

Sistema de Gestión de Datos de Países

Trabajo Práctico Integrador

Estudiante:	Lautaro Ezequiel Pérez
Institución:	Universidad Tecnológica Nacional (UTN)
Comisión:	N° 7
Fecha de entrega:	29 de junio de 2026

Índice General

1. Introducción y Objetivo
2. Marco Teórico
 - 2.1 Listas
 - 2.2 Diccionarios
 - 2.3 Funciones
 - 2.4 Estructuras Condicionales y Repetitivas
 - 2.5 Ordenamientos
 - 2.6 Estadísticas Básicas
 - 2.7 Archivos CSV
3. Decisiones Técnicas y Arquitectura
 - 3.1 Estructura de Módulos
 - 3.2 Flujo de Operaciones Principales
 - 3.3 Diagrama de flujo hecho en draw.io
 - 3.4 Descripción de funciones por módulo
 - 3.5 Capturas de pantalla
4. Dificultades y Conclusiones
5. Bibliografía / Webgrafía

1. Introducción y Objetivo

El presente trabajo práctico integrador corresponde a la materia Programación 1 de la Tecnicatura Universitaria en Programación a Distancia (TUPaD). El objetivo principal del proyecto fue desarrollar una aplicación en Python 3 que permita gestionar información sobre países, aplicando los conceptos aprendidos durante la cursada, como listas, diccionarios, funciones, estructuras condicionales, ciclos repetitivos, ordenamientos, estadísticas y manejo de archivos CSV.

El sistema desarrollado funciona mediante un menú interactivo en consola, desde el cual el usuario puede realizar distintas operaciones, como agregar nuevos países, buscar información, actualizar datos existentes, aplicar filtros, ordenar registros y consultar estadísticas generales.

La información se almacena en un archivo CSV, permitiendo conservar los cambios realizados y recuperar los datos cada vez que se ejecuta nuevamente el programa.

2. Marco Teórico

Durante el desarrollo del sistema se aplicaron diferentes conceptos fundamentales de programación en Python. Estos conceptos permitieron organizar la información, dividir las responsabilidades del código y lograr una estructura más clara y fácil de mantener:

2.1 Listas

Una lista en Python es una estructura de datos que permite almacenar varios elementos dentro de una misma variable. Es ordenada y modificable, lo que significa que sus valores pueden consultarse, agregarse o cambiarse durante la ejecución del programa.

En este proyecto, la lista principal utilizada fue `lista_paises`, donde se almacenan los datos de cada país. Cada elemento de la lista corresponde a un diccionario con la información del país.

```
lista_paises = []                # Inicialización de la lista vacía
lista_paises.append(pais)        # Inserción dinámica de un registro de país al
final cantidad = len(lista_paises) # Obtención del total de elementos
registrados
```

2.2 Diccionarios

Los diccionarios permiten almacenar información utilizando una relación entre claves y valores. A diferencia de una lista, donde los datos se buscan mediante posiciones numéricas, en un diccionario se accede mediante nombres definidos.

En el sistema, cada país se representa como un diccionario que contiene sus principales datos: nombre, población, superficie y continente. Esta estructura facilita la lectura y modificación de la información.

```
pais = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}  
print(pais["nombre"]) # Salida directa: Argentina
```

2.3 Funciones

Las funciones permiten dividir el programa en bloques de código independientes que realizan tareas específicas. Esto ayuda a mantener el código organizado y evita repetir instrucciones.

Durante el desarrollo se aplicó el principio de que cada función tenga una responsabilidad determinada. Por ejemplo, existen funciones encargadas de manejar archivos, otras para gestionar países y otras para calcular estadísticas.

2.4 Estructuras Condicionales y Repetitivas

Las estructuras condicionales permiten que el programa tome decisiones según diferentes situaciones utilizando instrucciones como `if`, `elif` y `else`.

Por otro lado, las estructuras repetitivas como `for` y `while` permiten recorrer datos o repetir acciones mientras se cumpla una condición.

En este proyecto, el menú principal funciona mediante un ciclo `while`, permitiendo que el usuario pueda realizar varias operaciones sin reiniciar el programa. También se utilizan ciclos para recorrer la lista de países, realizar búsquedas, aplicar filtros y calcular estadísticas.

2.5 Ordenamientos

Python cuenta con herramientas integradas para ordenar información. En este caso se utilizó la función `sorted()` para ordenar la lista de países según diferentes criterios, como población o superficie.

Mediante el uso del parámetro `key` y funciones `lambda` se puede indicar qué dato debe utilizarse como criterio de ordenamiento, permitiendo mostrar los resultados de forma ascendente o descendente.

```
# Ordenamiento de países por volumen poblacional de mayor a menor  
países_ordenados = sorted(lista_países, key=lambda p: p['poblacion'], reverse=True)
```

2.6 Estadísticas Básicas

El módulo `estadisticas.py` contiene funciones encargadas de analizar los datos almacenados.

Estas funciones permiten obtener información como el país con mayor población, el país con menor población, los promedios de población y superficie, y la cantidad de países pertenecientes a cada continente.

Las operaciones fueron desarrolladas utilizando la lógica aprendida durante la materia, sin depender de librerías externas.

Función Analítica	Descripción del Algoritmo e Impacto
<code>pais_mayor_poblacion()</code>	Recorre de forma lineal la lista de países, comparando los valores asociados a la clave de población para identificar y retornar el diccionario con el valor máximo.
<code>pais_menor_poblacion()</code>	Evalúa iterativamente la lista para determinar y devolver el registro de país que posea el menor volumen de habitantes acumulado.
<code>promedio_poblacion()</code>	Calcula la sumatoria matemática de la población de todos los países registrados y la divide por la cantidad total de elementos (<i>len</i>).
<code>promedio_superficie()</code>	Realiza de forma análoga la acumulación de las superficies declaradas, computando el promedio aritmético global del dataset.
<code>cantidad_por_continente()</code>	Construye y actualiza dinámicamente un diccionario contador (estructura clave-valor de tipo continente: cantidad) clasificando los países del sistema.

2.7 Archivos CSV

Los archivos CSV permiten almacenar información organizada en filas y columnas. Python incluye herramientas que facilitan la lectura y escritura de este tipo de archivos.

En el proyecto se utiliza el módulo `csv` junto con `DictReader` y `DictWriter`, permitiendo convertir automáticamente los registros del archivo en diccionarios y guardar nuevamente los cambios realizados.

```

# Mecanismo estructurado de Lectura
with open("países.csv", "r", encoding="utf-8") as f:
    lector = csv.DictReader(f)
    for fila in lector:
        print(fila["nombre"]) # Mapeo automático de campos

# Mecanismo estructurado de Escritura / Persistencia
with open("países.csv", "w", encoding="utf-8", newline="") as f:
    escritor = csv.DictWriter(f, fieldnames=["nombre", "poblacion", "superficie", "continente"])
    escritor.writeheader()
    escritor.writerow(pais)

```

3. Decisiones Técnicas y Arquitectura

3.1 Estructura de Módulos

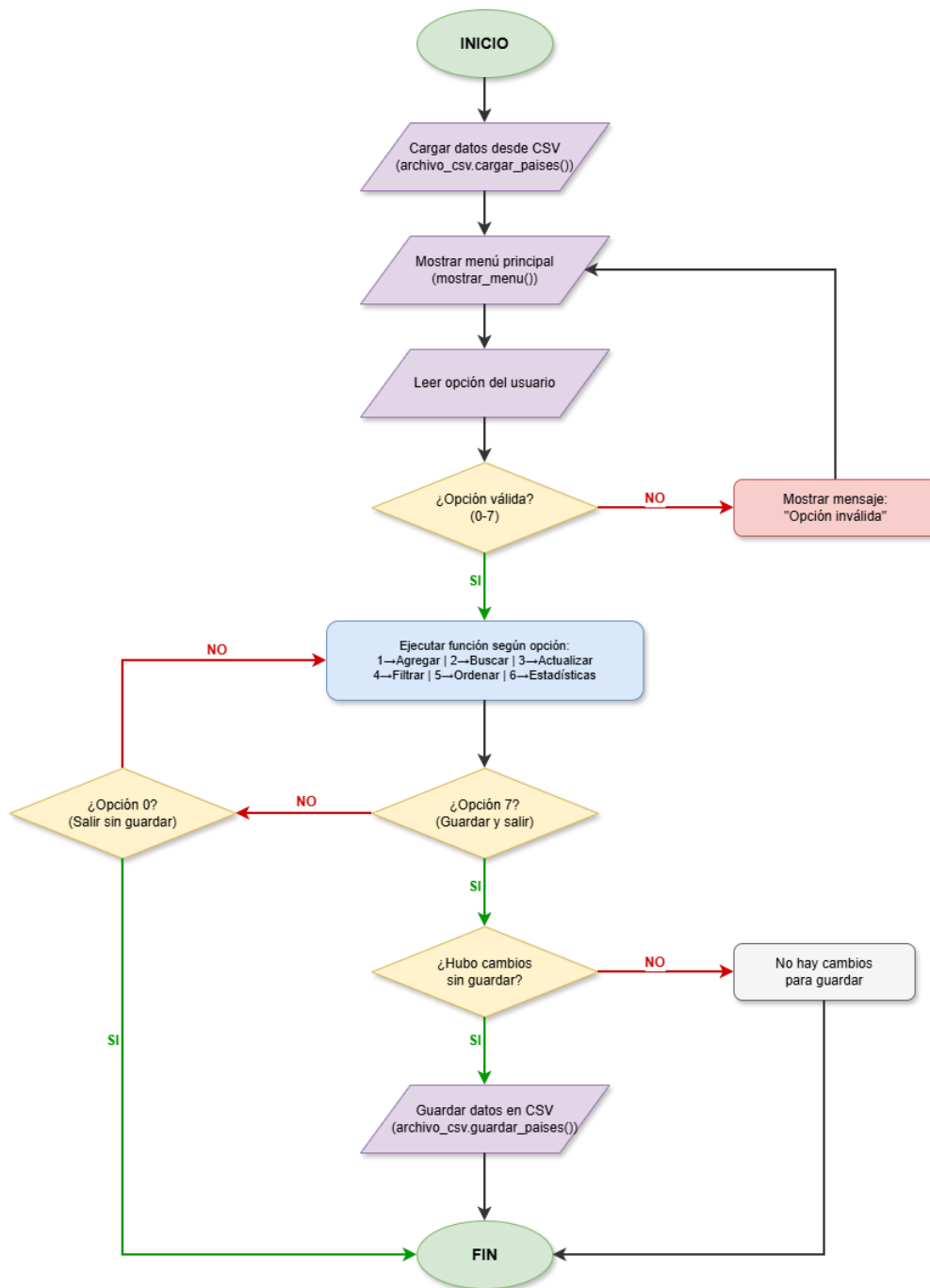
Para organizar el proyecto se decidió dividir el programa en diferentes módulos, separando las responsabilidades de cada parte del sistema.

Componente / Archivo	Responsabilidad Arquitectónica y Acoplamiento
main.py	Funciona como punto de inicio del programa y se encarga de mostrar el menú principal y coordinar las operaciones.
archivo_csv.py	Se ocupa exclusivamente de la lectura y escritura del archivo CSV, permitiendo cargar los datos al iniciar el programa y guardarlos cuando sea necesario.
gestion_paises.py	Contiene la lógica principal relacionada con los países, incluyendo funciones para agregar, buscar, actualizar, filtrar y ordenar registros.
estadisticas.py	Contiene las funciones encargadas de procesar los datos y obtener diferentes resultados estadísticos.
países.csv	Almacenamiento persistente que persiste en el disco y que contiene el dataset base en formato plano.

3.2 Flujo de Operaciones Principales

1. Al iniciar la aplicación, el programa carga los datos almacenados en el archivo CSV y crea la lista de países en memoria.
2. Luego se muestra un menú donde el usuario puede elegir qué operación desea realizar. Dependiendo de la opción seleccionada, el programa ejecuta la función correspondiente.
3. Todas las entradas ingresadas por el usuario cuentan con validaciones para evitar errores y mantener la información correcta.
4. Cuando el usuario decide guardar los cambios, el sistema actualiza el archivo CSV con los nuevos datos.

3.3 Diagrama de flujo hecho en draw.io:



3.4 Descripción de Funciones por Módulo

Módulo: `archivo_csv.py`

Signatura de la Función	Especificación Técnica
<i><code>cargar_paises(ruta_csv)</code></i>	Abre el archivo indicado con codificación UTF-8. Utiliza <i>DictReader</i> , realiza la conversión de tipos (casteo de cadenas a enteros para población y superficie) y genera la lista base.
<i><code>guardar_paises(lista_paises, ruta_csv)</code></i>	Serializa los diccionarios vigentes de la lista y los escribe en formato CSV plano. Incorpora bloques de captura para prevenir fallos de permisos en disco.

Módulo: `gestion_paises.py`

Signatura de la Función	Especificación Técnica
<i><code>agregar_pais(lista_paises)</code></i>	Solicita los campos interactivos implementando validaciones exhaustivas. Impide el ingreso de campos vacíos, tipos de datos incompatibles o claves de nombres repetidas.
<i><code>buscar_pais(lista_paises, termino)</code></i>	Ejecuta un escaneo lineal evaluando subcadenas parciales en el campo nombre, operando de manera insensible a mayúsculas y minúsculas (case-insensitive).
<i><code>actualizar_pais(pais)</code></i>	Modifica los atributos editables (población y superficie) directamente sobre el diccionario seleccionado. Al operar por referencia, los cambios impactan globalmente.
<i><code>filtrar_por_continente(lista, continente)</code></i>	Retorna una sublista filtrada con los países pertenecientes al continente especificado.
<i><code>filtrar_por_poblacion(lista, min, max)</code></i>	Filtra registros por umbrales poblacionales. Soporta límites abiertos enviando valores <i>None</i> en los extremos de la condición.

<i>filtrar_por_superficie(lista, min, max)</i>	Aplica de forma idéntica el criterio de discriminación por rangos acotados a los valores de la superficie terrestre.
<i>ordenar_paises(lista, clave, reverse)</i>	Genera una nueva vista ordenada de los datos sin alterar la colección original mediante la función integrada <i>sorted()</i> .
Signatura de la Función	Especificación Técnica
<i>mostrar_resultados_filtro(resultados)</i>	Homogeneiza la salida por pantalla, formateando las columnas y organizando la información tabulada para su correcta lectura.

Módulo: estadisticas.py

Signatura de la Función	Especificación Técnica
<i>pais_mayor_poblacion(lista_paises)</i>	Algoritmo de búsqueda de máximos mediante recorrido lineal sobre el campo de población.
<i>pais_menor_poblacion(lista_paises)</i>	Algoritmo de búsqueda de mínimos sobre el campo poblacional.
<i>promedio_poblacion(lista_paises)</i>	Acumulador de habitantes y división por longitud de lista. Retorna un valor flotante.
<i>promedio_superficie(lista_paises)</i>	Acumulador de kilómetros cuadrados totales y cálculo del promedio global.
<i>cantidad_por_continente(lista_paises)</i>	Genera un mapeo de frecuencias absolutas utilizando un diccionario auxiliar de control de continentes.
<i>mostrar_todas_estadisticas(lista)</i>	Función coordinadora que orquesta las llamadas analíticas y compone un reporte estético e integrado en la consola.

3.5 Capturas de pantalla

Inicio: Menú interactivo

```
=====
INICIANDO SISTEMA DE GESTIÓN DE PAÍSES
=====
```

```
Se cargaron 3 países desde el archivo.
```

```
=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
```

1. Agregar país
 2. Buscar país por nombre
 3. Actualizar población y superficie
 4. Filtrar países
 5. Ordenar países
 6. Mostrar estadísticas
 7. Guardar y salir
 0. Salir sin guardar
- ```
=====
```

```
Elegí una opción:
```

#### Opción 1: agregar país

```
Elegí una opción: 1
```

```
--- Agregar nuevo país ---
```

```
Nombre del país:
```

```
Error: el nombre no puede estar vacío.
```

```
Nombre del país: Chile
```

```
Población: asdas
```

```
Error: ingresá un número entero válido para la población.
```

```
Población: 19600000
```

```
Superficie (km^2): 756102
```

```
Continente: América
```

```
País 'Chile' agregado correctamente.
```

#### Opción 2: buscar país por nombre

```
Elegí una opción: 2
```

```
Ingresá el nombre o parte del nombre a buscar: Ch
```

```
--- Resultados encontrados (1) ---
```

```
1. Chile - Población: 19,600,000 - Superficie: 756,102 km² - Continente: América
```

#### Opción 3: actualizar población y superficie

Elegí una opción: 3

--- Actualizar país ---

Primero buscá el país que querés modificar:

Ingresá el nombre o parte del nombre a buscar: Chile

--- Resultados encontrados (1) ---

1. Chile - Población: 19,600,000 - Superficie: 756,102 km<sup>2</sup> - Continente: América

0. Cancelar

Seleccioná el país (1-1): 1

--- Actualizar país ---

Datos actuales de 'Chile':

Población: 19600000

Superficie: 756102 km<sup>2</sup>

(Dejá el campo vacío para mantener el valor actual)

Nueva población: 20000000

Nueva superficie (km<sup>2</sup>):

País 'Chile' actualizado correctamente.

#### Opción 4: filtrar países (por continente)

Elegí una opción: 4

--- FILTRAR PAÍSES ---

1. Filtrar por continente
2. Filtrar por rango de población
3. Filtrar por rango de superficie
0. Volver al menú principal

Elegí una opción: 1

Ingresá el continente: América

--- Resultados del filtro: Continente: América (2 países) ---

1. Chile - Población: 20,000,000 - Superficie: 756,102 km<sup>2</sup> - Continente: América
2. Brasil - Población: 213,500,000 - Superficie: 8,515,767 km<sup>2</sup> - Continente: América

--- FILTRAR PAÍSES ---

1. Filtrar por continente
2. Filtrar por rango de población
3. Filtrar por rango de superficie
0. Volver al menú principal

Elegí una opción: █

#### Opción 4: filtrar países (por población)

Elegí una opción: 2

Población mínima (Enter para omitir): 50000000

Población máxima (Enter para omitir): 300000000

--- Resultados del filtro: Rango de población (2 países) ---

1. Japón - Población: 122,400,000 - Superficie: 377,975 km<sup>2</sup> - Continente: Asia
2. Brasil - Población: 213,500,000 - Superficie: 8,515,767 km<sup>2</sup> - Continente: América

--- FILTRAR PAÍSES ---

1. Filtrar por continente
2. Filtrar por rango de población
3. Filtrar por rango de superficie
0. Volver al menú principal

Elegí una opción: █

#### Opción 4: filtrar países (por superficie)

```
Elegí una opción: 3
Superficie mínima (km²) (Enter para omitir): 300000
Superficie máxima (km²) (Enter para omitir): 1000000

--- Resultados del filtro: Rango de superficie (2 países) ---
1. Chile - Población: 20,000,000 - Superficie: 756,102 km^2 - Continente: América
2. Japón - Población: 122,400,000 - Superficie: 377,975 km^2 - Continente: Asia

--- FILTRAR PAÍSES ---
1. Filtrar por continente
2. Filtrar por rango de población
3. Filtrar por rango de superficie
0. Volver al menú principal

Elegí una opción: █
```

#### Opción 5: ordenar países

```
Elegí una opción: 5

--- ORDENAR PAÍSES ---
1. Ordenar por nombre
2. Ordenar por población
3. Ordenar por superficie

Elegí un criterio: 2
¿Orden ascendente? (s/n): n

--- Países ordenados por poblacion (descendente) ---
1. Brasil - Población: 213,500,000
2. Japón - Población: 122,400,000
3. Chile - Población: 20,000,000
```

#### Opción 6: mostrar estadísticas

```
Elegí una opción: 6

=====
ESTADÍSTICAS DE PAÍSES
=====

- País con MAYOR población:
 Brasil - 213,500,000 habitantes

- País con MENOR población:
 Chile - 20,000,000 habitantes

- Promedio de población: 118,633,333 habitantes

- Promedio de superficie: 3,216,615 km²

- Cantidad de países por continente:
 América: 2 país(es)
 Asia: 1 país(es)
```

#### Opción 7: Guardar y salir

```
Elegí una opción: 7
Datos guardados correctamente en 'países.csv'.
¡Hasta luego!
```

#### Opción 0: Salir sin guardar

```
Elegí una opción: 0
¡Hasta luego!
```

## 4. Dificultades y Conclusiones

### Dificultades Encontradas

Una de las principales dificultades durante el desarrollo fue comprender correctamente cómo Python trabaja con los diccionarios cuando se utilizan dentro de funciones.

Al pasar un diccionario como argumento, la función recibe una referencia al mismo objeto, por lo que cualquier modificación realizada dentro de ella afecta directamente al dato original. Comprender este comportamiento fue importante para implementar correctamente la función de actualización de países y evitar trabajar con copias que no modificaran los datos reales.

Otra dificultad importante fue organizar correctamente la estructura del proyecto. Separar las funciones en diferentes módulos requirió planificar qué responsabilidad debía tener cada archivo para evitar dependencias innecesarias y mantener el código ordenado.

## Conclusiones

El desarrollo de este sistema permitió aplicar de forma práctica los conceptos aprendidos durante la materia. Trabajar con listas de diccionarios resultó una forma adecuada de representar los datos de los países, mientras que el uso de archivos CSV permitió agregar persistencia a la información.

La separación del código en módulos facilitó la organización y el mantenimiento del programa, ya que cada parte del sistema tiene una función específica. Esto permitió detectar errores con mayor facilidad y comprender mejor cómo estructurar una aplicación completa.

En conclusión, el proyecto permitió integrar los principales conceptos de programación vistos durante la cursada y aplicarlos en una solución funcional, organizada y mantenible.

## 5. Bibliografía

---

- [1] **Python Software Foundation** - Documentación oficial de Python 3: Módulo nativo de procesamiento csv.  
<https://docs.python.org/3/library/csv.html>
- [2] **Python Software Foundation** - Documentación oficial de Python 3: Manejo estructurado de errores y excepciones.  
<https://docs.python.org/3/tutorial/errors.html>

## 6. Anexos

---

- [1] **Repositorio del Proyecto en GitHub** - Código fuente, dataset inicial CSV y documentación técnica complementaria (README).  
<https://github.com/SoyEZZEK/UTN-P1-TPI>
- [2] **Video Demostrativo del Programa** - Grabación de la explicación y ejemplo de funcionamiento del programa.  
<https://youtu.be/rmkVMoHBpsA>